



南開大學
Nankai University

计算机学院
并行程序设计期末报告

并行程序设计 **NTT** 期末综合报告

姓名：强徽东

学号：2411764

专业：计算机科学与技术

2026 年 7 月 12 日

目录

I	NTT 共同算法基础	2
1	实验背景、研究目标与材料来源	2
1.1	github 地址	2
1.2	问题背景	2
1.3	平时作业与新增内容标记	2
2	NTT 数学原理与串行基准	2
2.1	单位根与正逆变换	3
2.2	蝶形分解	3
2.3	串行 NTT 基准代码	3
2.4	位逆序与 DIT/DIF	4
3	模乘规约与 CRT 大模数处理	5
3.1	普通模乘的瓶颈	5
3.2	Montgomery 规约及代码	5
3.3	Barrett 规约	5
3.4	Lazy Reduction 延迟规约	6
3.5	CRT/Garner 合并	7
4	统一 NTT 计算图与跨架构映射	7
4.1	统一计算层次	7
4.2	映射、同步与主要瓶颈	8
II	不同体系结构上的并行优化	8
5	SIMD 指令级优化	8
5.1	NEON 向量化对象	8
5.2	四路 Montgomery 模乘	8
5.3	向量化蝶形	9
5.4	DIT/DIF 与 SIMD 的联合效果	9
6	Pthread 共享内存并行	9
6.1	线程共享上下文	9
6.2	两种 stage 划分方式	10
6.3	主线程参与和线程生命周期	10
6.4	CRT 模数并行与合并并行	11
7	OpenMP 自动调度与 SIMD 向量化	11
7.1	OpenMP NTT 内部并行	11

7.2	OpenMP CRT 模数并行	11
7.3	CRT 合并的调度策略	11
7.4	旋转因子预计算与 omp simd	12
7.5	Pthread、OpenMP 与 OpenMP SIMD 对比	12
8	MPI 分布式并行	12
8.1	输入广播和计时方式	12
8.2	CRT 模数级 MPI 并行	13
8.3	NTT stage 级 MPI 并行	13
8.4	Barrett、DIT/DIF 与混合同行	14
9	GPU 并行优化：HIP 实验	14
9.1	一个线程负责一个蝶形	14
9.2	旋转因子预计算	14
9.3	GPU Barrett 与 Montgomery	14
9.4	LDS 分块	15
III	在共同算法与并行优化基础上的新增内容	15
10	统一 NTT 算法框架与实验基准	15
10.1	计划生成与执行分离	15
10.2	统一后端抽象	16
10.3	统一卷积流程	16
10.4	统一结果格式和基本正确性保证	16
11	跨层次性能开销模型	16
11.1	统一总时间分解	17
11.2	SIMD 与共享内存线程模型	17
11.3	MPI 两种粒度模型	17
11.4	GPU 端到端模型	17
12	跨架构评价指标设计	18
12.1	为什么不能直接排列原始时间	18
12.2	归一化指标	18
12.3	消融实验逻辑	18
IV	统一性能测试与综合分析	18
13	实验环境与统一测试方法	19
13.1	各平台环境	19
13.2	统一实验规模与输入	19
13.3	计时边界	19

13.4 重复实验与命令	19
14 各体系结构内部性能分析	20
14.1 SIMD 版本消融实验	20
14.2 Pthread/OpenMP 版本消融实验	20
14.3 MPI 版本消融实验	21
14.4 HIP 版本性能分析	21
15 跨架构综合分析	22
15.1 跨机器原始时间的可比性	22
15.2 归一化性能总结	22
15.3 规模扩展性	22
15.4 优化不能线性叠加的原因	22
15.5 基于问题特征的 NTT 后端选择策略	22
16 实验局限与改进方向	23
17 新增内容说明	24
18 总结	24

Part I

NTT 共同算法基础

1 实验背景、研究目标与材料来源

1.1 github 地址

实验相关内容可在 <https://github.com/huiqwq/-> 查看

1.2 问题背景

给定两个长度为 n 的多项式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i,$$

需要计算

$$C(x) = A(x)B(x) = \sum_{k=0}^{2n-2} c_k x^k, \quad c_k = \sum_{i+j=k} a_i b_j \pmod{P}.$$

朴素实现直接枚举 (i, j) , 代码如清单 1.2 所示。

朴素多项式卷积

```

1 void poly_multiply(int *a, int *b, int *ab, int n, int p) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             ab[i + j] = (1LL * a[i] * b[j] % p
5                 + ab[i + j]) % p;
6         }
7     }
8 }
```

该代码包含 n^2 次乘加, 复杂度为 $O(n^2)$ 。当 $n = 131072$ 时, 直接卷积的计算量已无法接受。NTT 把多项式从系数表示变换到点值表示, 在点值域逐点相乘后再逆变换, 将复杂度降低为 $O(n \log n)$ 。由于同一 stage 内的蝶形互不冲突, NTT 也天然适合多层次并行。

1.3 平时作业与新增内容标记

全文使用【平时作业】、【融合重构】和【本次新增】三种标记。前者表示代码、实验或数据来自平时作业; 第二种表示旧材料已按统一问题重新组织; 第三种表示本次新增的框架、代码或综合分析。

2 NTT 数学原理与串行基准

【平时作业】 【融合重构】

2.1 单位根与正逆变换

在质数模数 p 下, 若 $N \mid (p-1)$, 可由原根 g 得到 N 次单位根

$$\omega_N = g^{(p-1)/N} \pmod{p}.$$

NTT 定义为

$$A_k = \sum_{j=0}^{N-1} a_j \omega_N^{jk} \pmod{p}.$$

逆变换将单位根换成 ω_N^{-1} , 并在最后乘以 $N^{-1} \pmod{p}$. 常用模数

$$998244353 = 119 \times 2^{23} + 1$$

支持最大长度 2^{23} 的 radix-2 NTT, 原根通常取 3。

2.2 蝶形分解

把多项式按偶、奇下标分解后, 每层蝶形可写为

$$u = a_{base+j}, \quad v = a_{base+j+L/2} \omega_L^j \pmod{p},$$

$$a_{base+j} = u + v \pmod{p}, \quad a_{base+j+L/2} = u - v \pmod{p}.$$

长度为 N 的每个 stage 都包含 $N/2$ 个蝶形; 同一 stage 内蝶形访问互不重叠, 但后一 stage 必须等待前一 stage 完成。这一性质决定了 Pthread barrier、OpenMP 隐式 barrier、MPI 集合通信和 GPU kernel 边界的位置。

2.3 串行 NTT 基准代码

清单 2.3 给出报告各后端共享的串行逻辑。代码先位逆序, 再让 `len` 从 2 逐层翻倍; 每层的 `wn` 是该层单位根。

串行迭代 NTT 核心

```

1 void ntt_serial(vector<u64>& f, u64 mod,
2               u64 root, bool inverse) {
3     int n = (int)f.size();
4     bit_reverse(f);
5     for (int len = 2; len <= n; len <= 1) {
6         u64 wn = qpow(root, (mod - 1) / len, mod);
7         if (inverse) wn = qpow(wn, mod - 2, mod);
8         int half = len >> 1;
9         for (int base = 0; base < n; base += len) {
10             u64 w = 1;
11             for (int j = 0; j < half; ++j) {
12                 u64 u = f[base + j];
13                 u64 v = mul_mod(f[base + j + half], w, mod);
14                 f[base + j] = add_mod(u, v, mod);
15                 f[base + j + half] = sub_mod(u, v, mod);
16                 w = mul_mod(w, wn, mod);
17             }
18         }

```

```

19     }
20     if (inverse) {
21         u64 inv_n = qpow(n, mod - 2, mod);
22         for (u64& x : f) x = mul_mod(x, inv_n, mod);
23     }
24 }

```

该实现也是正确性和性能的基础参照。后续各并行版本只改变 `base/j` 循环如何分配以及 `mul_mod` 如何实现，不改变蝶形的数学结果。

2.4 位逆序与 DIT/DIF

迭代 DIT 需要在变换开始前进行位逆序。位逆序由大量不连续交换组成，对 cache 和 GPU 全局内存都不友好。平时 SIMD 与 MPI 实验进一步采用 DIF 正变换与 DIT 逆变换配对：正变换产生位逆序输出，点值乘不依赖排列顺序，逆变换直接接收该顺序，从而省去卷积流程中的显式重排。

DIT 和 DIF 的主要差别是旋转因子与加减运算的先后顺序。DIF 正变换先完成模加减，再将差值乘以旋转因子，并且 stage 长度由 N 逐层减小：

DIF 正变换核心代码

```

1 void ntt_dif_forward(vector<u64>& f, u64 mod, u64 root) {
2     int n = (int)f.size();
3     for (int len = n; len >= 2; len >>= 1) {
4         int half = len >> 1;
5         u64 wn = qpow(root, (mod - 1) / len, mod);
6         for (int base = 0; base < n; base += len) {
7             u64 w = 1;
8             for (int j = 0; j < half; ++j) {
9                 u64 x = f[base + j];
10                u64 y = f[base + j + half];
11                f[base + j] = add_mod(x, y, mod);
12                f[base + j + half] =
13                    mul_mod(sub_mod(x, y, mod), w, mod);
14                w = mul_mod(w, wn, mod);
15            }
16        }
17    }
18 }

```

与之配对的 DIT 逆变换按相反方向增大 stage，先将右半部乘以逆旋转因子，再执行加减：

DIT 逆变换核心代码

```

1 void intt_dit_inverse(vector<u64>& f, u64 mod, u64 root) {
2     int n = (int)f.size();
3     for (int len = 2; len <= n; len <<= 1) {
4         int half = len >> 1;
5         u64 wn = qpow(root, (mod - 1) / len, mod);
6         wn = qpow(wn, mod - 2, mod);
7         for (int base = 0; base < n; base += len) {
8             u64 w = 1;
9             for (int j = 0; j < half; ++j) {
10                u64 x = f[base + j];
11                u64 y = mul_mod(f[base + j + half], w, mod);
12                f[base + j] = add_mod(x, y, mod);
13                f[base + j + half] = sub_mod(x, y, mod);

```

```

14         w = mul_mod(w, wn, mod);
15     }
16 }
17 }
18 u64 inv_n = qpow((u64)n, mod - 2, mod);
19 for (u64& x : f) x = mul_mod(x, inv_n, mod);
20 }

```

两次 DIF 正变换的输出都是相同的位逆序排列，因此可以直接逐点相乘；DIT 逆变换恰好接收这种排列，最后再统一乘以 N^{-1} 。代码中因此不需要调用独立的位逆序交换函数。

需要强调的是，DIT/DIF 的价值不是改变 $O(N \log N)$ 复杂度，而是减少额外交换和不连续访存。它在 CPU、MPI 本地 NTT 与 GPU 上具有相同算法意义。

3 模乘规约与 CRT 大模数处理

【平时作业】 【融合重构】

3.1 普通模乘的瓶颈

NTT 蝶形中最频繁的运算是

$$v = a \cdot w \bmod p.$$

普通 $a \cdot b \bmod p$ 可能生成整数除法：当 p 接近 10^9 时，32 位操作数的中间乘积已经需要 64 位；更大模数还可能 128 位中间结果。因此，规约方式既影响正确性，也影响 SIMD 和 GPU 是否容易实现。

3.2 Montgomery 规约及代码

取 $R = 2^{32}$ ，令

$$p' = -p^{-1} \bmod R.$$

Montgomery 规约计算 $TR^{-1} \bmod p$ ，把除以 R 转化为右移。平时 SIMD 实验中的标量代码如下。

32 位 Montgomery 规约

```

1 u32 reduce(u64 x) const {
2     u32 q = (u32)((x & 0xffffffffULL) * inv);
3     u64 y = (x + u64(q) * mod) >> 32;
4     if (y >= mod) y -= mod;
5     return u32(y);
6 }

```

第 2 行只保留 x 的低 32 位并乘以 p' ；根据 p' 的定义， $x + qp$ 一定是 2^{32} 的倍数，因此第 3 行可用右移完成除法。最后一次条件减法把结果调整到 $[0, p)$ 。当输入、旋转因子和 N^{-1} 在整次变换中都保持为 Montgomery 表示时，进入域与退出域的固定成本只支付一次。

3.3 Barrett 规约

Barrett 规约的核心是用一次预计算的乘法倒数代替每次模乘中的整数除法。对固定模数 p ，预计算

$$\mu = \left\lfloor \frac{2^{64}}{p} \right\rfloor,$$

对 $x = ab < p^2$ ，利用乘法高位估计商

$$\hat{q} = \left\lfloor \frac{x\mu}{2^{64}} \right\rfloor, \quad r = x - \hat{q}p.$$

由于 $\hat{q}$ 只是近似商， r 可能仍大于模数，最后需要一到两次条件减法把结果调整到 $[0, p)$ 。平时 MPI 代码的实现如下：

Barrett 规约与模乘

```

1 struct BarrettReducer { u64 mod, mu; };
2 BarrettReducer make_barrett(u64 mod) {
3     u64 mu = (u64)((u128)1 << 64) / mod;
4     return {mod, mu};
5 }
6 u64 barrett_reduce(u64 x, const BarrettReducer& br) {
7     u64 q = (u64)((u128)x * br.mu >> 64);
8     u64 r = x - q * br.mod;
9     if (r >= br.mod) r -= br.mod;
10    if (r >= br.mod) r -= br.mod;
11    return r;
12 }
13 u64 mul_mod_barrett(u64 a, u64 b, const BarrettReducer& br) {
14     return barrett_reduce(a * b, br);
15 }

```

代码中的宽整数乘法用于取 $x\mu$ 的高 64 位，不再执行 “ $x \% \text{mod}$ ” 对应的整数除法。对本实验使用的 32 位 NTT 友好模数， ab 可以放入 64 位无符号整数。HIP 版本中的高位乘法指令与这一步等价，因此同一思路可同时用于 CPU 和 GPU。

MPI v3 和 HIP 实验都使用了 Barrett。它能降低本地模乘开销，但若 MPI 主要受通信限制，或完整 HIP NTT 主要受访存和 kernel 启动限制，局部模乘的加速并不会按相同比例反映到总时间上。

3.4 Lazy Reduction 延迟规约

【本次新增】

普通蝶形会把每个输出立即规约到 $[0, p)$ ；Lazy Reduction 则允许中间值暂时保留在更大的等价区间，以减少条件减法次数。它与 Barrett/Montgomery 并不冲突：前者减少规约次数，后两者降低单次模乘规约的成本。

HIP 单模数实验使用 $p = 998244353$ ，满足 $4p < 2^{32}$ ，因此可在 32 位无符号整数中维持 $[0, 4p)$ 值域。设输入 $x, y < 4p$ ，先把 x 调整到 $[0, 2p)$ ，再令 $t = yw \bmod p < p$ ，则 $x + t < 3p$ 且 $x + 2p - t < 4p$ ：

适用于 998244353 的 Lazy 蝶形

```

1 __device__ inline void lazy_butterfly(u32& x, u32& y, u32 w) {
2     constexpr u32 TWO_P = 2 * MOD;
3     u32 u = x;
4     if (u >= TWO_P) u -= TWO_P;
5     u32 t = barrett_mul(y, w); // t in [0, p)
6     x = u + t;                 // x in [0, 3p)
7     y = u + TWO_P - t;        // y in [0, 4p)
8 }
9 u32 normalize(u32 x) {
10    constexpr u32 TWO_P = 2 * MOD;
11    if (x >= TWO_P) x -= TWO_P;
12    return x >= MOD ? x - MOD : x;

```

13 }

整个 NTT 期间保持扩展值域，点值乘和逆变换缩放后再按需要统一标准化。该实现只对满足 $4p < 2^{32}$ 的模数安全；四模数 CRT 中的较大模数不满足这一条件，服务器复测应改用 64 位中间数组，或将允许范围缩小到 $[0, 2p)$ 。因此 Lazy Reduction 的核心不仅是删除条件减法，还包括对每个模数和数据类型进行明确的溢出界证明。

3.5 CRT/Garner 合并

当目标模数 P 不是 NTT 友好模数，或 P 很大时，程序在四个两两互质的小模数下分别计算 residue，再用增量 CRT 合并。

增量 CRT 合并一个输出系数

```

1 u64 crt_combine_one(const u64 r[CRT_CNT],
2                     u64 target_mod,
3                     const u64 inv_prod_mod[CRT_CNT]) {
4     u128 x = r[0];
5     u128 prod = CRT_MODS[0].mod;
6
7     for (int i = 1; i < CRT_CNT; ++i) {
8         u64 mi = CRT_MODS[i].mod;
9         u64 cur = (u64)(x % mi);
10        u64 need = (r[i] + mi - cur) % mi;
11        u64 t = mul_mod(need, inv_prod_mod[i], mi);
12        x += prod * t;
13        prod *= mi;
14    }
15    return (u64)(x % target_mod);
16 }
```

循环保持不变量 $x \equiv r_j \pmod{p_j}$ 。每加入一个模数，就求出使新同余成立的混合进制位 t 。u128 用于容纳四模数乘积。CRT 正确性的关键条件是

$$\prod_i p_i > n(C-1)^2,$$

其中 C 是输入系数上界。平时作业中曾因三模数或第四模数过小导致大模数样例错误，说明模数选择是数学正确性问题，而不是单纯性能参数。

4 统一 NTT 计算图与跨架构映射

【本次新增】

4.1 统一计算层次

把四次作业重新放入同一计算图，可以得到：

NTT 卷积的统一计算层次

```

1 batch / test case
2 -> CRT modulus p0 ... p3
3 -> forward(A), forward(B), pointwise, inverse
```

```

4   -> stage 0 ... logN-1      // inter-stage dependency
5   -> N/2 butterflies         // independent in one stage
6   -> mul, add, subtract modulo p

```

最内层模运算适合 SIMD；stage 内蝶形适合 Pthread/OpenMP 或 GPU；CRT 模数之间适合 OpenMP/MPI 粗粒度任务并行；多个批次又可作为更外层任务。融合的核心是选择哪一层交给哪种硬件，而不是把所有技术强行叠加在一个循环上。

4.2 映射、同步与主要瓶颈

表 1: NTT 在不同并行体系结构上的映射

层次	并行对象	数据共享	同步位置	主要开销
SIMD	连续 4/8 个蝶形	向量寄存器	指令内隐式	64 位乘积拆分、lane 重排
Pthread/ OpenMP MPI	stage 内蝶形区间 CRT 模数或蝶形块	主存与 cache 消息显式共享	每层 barrier Send/Recv Allgather kernel 边界	线程调度、同步、内存带宽 启动延迟和通信量
GPU	每线程一个蝶形	全局内存/LDS	kernel 边界	启动、全局访存、occupancy

表 1 给出了后文所有版本的统一比较坐标。SIMD 几乎没有显式同步，但受数据布局限制；Pthread/OpenMP 共享数组，通信代价低但每层必须同步；MPI 能扩展到多个进程，却必须显式传输数据；GPU 并行度高，但不同 block 的全局同步通常只能依靠 kernel 边界。

Part II

不同体系结构上的并行优化

5 SIMD 指令级优化

【平时作业】 【融合重构】

5.1 NEON 向量化对象

ARM NEON 寄存器宽度为 128 位，可一次容纳 4 个 32 位无符号整数。NTT 中适合向量化的操作包括蝶形模乘与模加减、点值乘法以及逆变换归一化。困难在于两个 32 位数的乘积需要 64 位中间结果，而一个 128 位寄存器只能容纳两个 64 位结果。

5.2 四路 Montgomery 模乘

平时 SIMD 代码把 4 路乘法拆成低两路和高两路：

NEON 四路乘法的低高两组拆分

```

1  uint64x2_t prod_low = vmull_u32(
2      vget_low_u32(a), vget_low_u32(b));
3  uint64x2_t prod_high = vmull_u32(
4      vget_high_u32(a), vget_high_u32(b));

```

```

5
6 uint32x2_t low = montgomery_reduce2_neon(
7     prod_low, mont.mod, mont.inv);
8 uint32x2_t high = montgomery_reduce2_neon(
9     prod_high, mont.mod, mont.inv);
10 return vcombine_u32(low, high);

```

前两行分别产生 $[a_0b_0, a_1b_1]$ 和 $[a_2b_2, a_3b_3]$ 。随后两次 Montgomery 规约把 64 位结果降回 32 位，最后合并成 4 路向量。这段代码说明 SIMD 宽度并不等于实际乘法吞吐：扩宽、拆分、规约与合并都属于额外指令。

5.3 向量化蝶形

得到 4 路旋转因子乘积后，模加和模减可以完全向量化：

NEON 向量化模加和模减

```

1 uint32x4_t roots = vld1q_u32(roots_buf);
2 uint32x4_t right = vld1q_u32(&f[base + j + half]);
3 uint32x4_t v = montgomery_mul4_neon(roots, right, mont);
4
5 uint32x4_t sum = vaddq_u32(u, v);
6 uint32x4_t ge = vcgeq_u32(sum, mod_v);
7 sum = vsubq_u32(sum, vandq_u32(ge, mod_v));
8
9 uint32x4_t diff = vsubq_u32(u, v);
10 uint32x4_t need = vcltq_u32(u, v);
11 diff = vaddq_u32(diff, vandq_u32(need, mod_v));

```

`vcgeq` 和 `vcltq` 生成逐 lane 掩码，再用按位与选择是否加减模数，从而避免分支。相同的四路 Montgomery 乘法还用于点值乘和逆变换缩放，使 SIMD 覆盖主要热点。

5.4 DIT/DIF 与 SIMD 的联合效果

平时结果中，单独把 Montgomery 模乘改成 NEON 并没有超过仅向量化加减的版本；原因是 64 位乘积的拆分与重组抵消了部分收益。加入 DIT/DIF 后平均时间进一步下降，说明减少位逆序访存比继续增加局部 SIMD 指令更有效。该结果体现了优化顺序：先消除不必要的移动，再压缩单个蝶形的算术成本。

6 Pthread 共享内存并行

【平时作业】 【融合重构】

6.1 线程共享上下文

Pthread 版本让所有线程共享 NTT 数组、模数和当前方向，并用 barrier 保证 stage 顺序：

Pthread NTT 共享上下文

```

1 struct NttContext {
2     u64 *f;
3     int n, threads;

```

```

4   u64 mod;
5   bool inverse;
6   pthread_barrier_t barrier;
7 };

```

上下文只保存一份数组指针，线程间无需复制数据。`threads` 同时用于任务划分和 `barrier` 参与者数量，因此主线程也必须执行工作函数，才能在 `barrier` 上与子线程汇合。

6.2 两种 stage 划分方式

NTT 前期 `len` 小、`block` 数多，适合按 `block` 轮转；后期 `block` 数少但每个 `block` 包含大量蝶形，需改为按蝶形总数划分。平时代码的选择逻辑如下：

根据 stage 形态切换 Pthread 粒度

```

1  for (int len = 2; len <= n; len <= 1) {
2      u64 wn = qpow(G, (mod - 1) / len, mod);
3      if (inverse) wn = qpow(wn, mod - 2, mod);
4
5      int blocks = n / len;
6      if (blocks >= threads)
7          ntt_stage_by_blocks(f, n, len, wn, mod, tid, threads);
8      else
9          ntt_stage_by_butterflies(f, n, len, wn, mod, tid, threads);
10
11     pthread_barrier_wait(&ctx->barrier);
12 }

```

若始终按完整 `block` 划分，则最后一层只有一个 `block`，大多数线程空闲。按蝶形总数切换后，即使 `len=n`，也能把 $n/2$ 个蝶形均分给线程。每层末尾的 `barrier` 不可省略，因为下一层会读取本层其他线程写入的数据。

6.3 主线程参与和线程生命周期

创建子线程并让主线程参与

```

1  int worker_threads = threads - 1;
2  for (int t = 0; t < worker_threads; ++t) {
3      tasks[t] = {&ctx, t};
4      pthread_create(&tids[t], nullptr, ntt_worker, &tasks[t]);
5  }
6
7  run_ntt_worker(&ctx, worker_threads); // main thread works
8
9  for (pthread_t tid : tids) pthread_join(tid, nullptr);

```

实验平台每个任务有 8 核，因此最多创建 7 个子线程并让主线程承担第 8 份任务。这样避免主线程只等待而浪费核心。Pthread 的优点是能精确控制生命周期、划分和 `barrier`；缺点是代码量与调试复杂度高。

6.4 CRT 模数并行与合并并行

四个小模数 NTT 完全独立，可以由 3 个子线程和主线程分别计算。所有 residue 完成后，输出系数之间也互不依赖，因此 CRT 合并可按连续区间再次并行。该粗粒度策略的同步次数远少于在每个 NTT stage 内建立线程，适合四模数大模数卷积。

7 OpenMP 自动调度与 SIMD 向量化

【平时作业】 【融合重构】

7.1 OpenMP NTT 内部并行

OpenMP 版本保留 Pthread 的粒度判断，但把显式线程创建替换为运行时调度。前期按 block 静态分配：

OpenMP 按 block 静态调度

```
1 #pragma omp parallel for schedule(static)
2 for (int block = 0; block < blocks; ++block) {
3     int base = block * len;
4     u64 w = 1;
5     for (int k = 0; k < half; ++k) {
6         u64 x = f[base + k];
7         u64 y = mul_mod(f[base + k + half], w, mod);
8         f[base + k] = add_mod(x, y, mod);
9         f[base + k + half] = sub_mod(x, y, mod);
10        w = mul_mod(w, wn, mod);
11    }
12 }
```

`schedule(static)` 适合每个 block 计算量相同的规则循环。循环末尾默认存在隐式 barrier，正好满足 stage 依赖。后期 block 数不足时，代码进入 parallel region，按全体蝶形的连续编号计算每个线程的区间。

7.2 OpenMP CRT 模数并行

四模数 OpenMP 任务并行

```
1 int mod_threads = min(get_omp_threads(), CRT_CNT);
2 #pragma omp parallel for schedule(static) num_threads(mod_threads)
3 for (int i = 0; i < CRT_CNT; ++i) {
4     multiply_mod_ntt(a, b, residues[i], n, CRT_MODS[i]);
5 }
```

该循环只有 4 个任务，因此最大并行度有限；但每个任务包含完整的两次正 NTT、点乘和逆 NTT，粒度很粗，运行时调度与同步开销相对较小。平时实验中它优于频繁进入并行区的内部 NTT 版本。

7.3 CRT 合并的调度策略

CRT 输出系数的 guided 调度

```

1 #pragma omp parallel for schedule(guided, 256)
2 for (int i = 0; i < result_len; ++i) {
3     u64 r[CRT_CNT];
4     for (int j = 0; j < CRT_CNT; ++j)
5         r[j] = residues[j][i];
6     ab[i] = crt_combine_one(r, target_mod, inv_prod_mod);
7 }

```

每个系数都执行相同数量的 Garner 步骤，理论上 `static` 已足够；平时代码选择 `guided` 是为了减小尾部不均衡。应把这视为调度策略对照，而不能默认动态调度总是更快，因为任务获取本身也有成本。

7.4 旋转因子预计算与 `omp simd`

原始蝶形使用 $w=w*wn$ ，相邻迭代存在循环携带依赖，编译器难以向量化。平时 OpenMP SIMD 版本先生成 `roots[k]`，再对蝶形加 `omp simd`：

消除旋转因子依赖后的 `omp simd`

```

1 vector<u64> roots(half);
2 roots[0] = 1;
3 for (int k = 1; k < half; ++k)
4     roots[k] = mul_mod_ntt(roots[k - 1], wn, mod);
5
6 for (int base = 0; base < n; base += len) {
7     #pragma omp simd
8     for (int k = 0; k < half; ++k) {
9         u64 x = f[base + k];
10        u64 y = mul_mod_ntt(f[base + k + half], roots[k], mod);
11        f[base + k] = add_mod(x, y, mod);
12        f[base + k + half] = sub_mod(x, y, mod);
13    }
14 }

```

预计算把递推关系移到循环外，使每个 `k` 只读自己的旋转因子。`omp simd` 是编译器提示，不保证一定生成最优向量指令；其效果依赖编译器能否处理模乘。

7.5 Pthread、OpenMP 与 OpenMP SIMD 对比

Pthread 提供最细的线程控制，适合自定义 stage 粒度和 barrier；OpenMP 代码更简洁，适合规则循环和粗粒度模数任务；OpenMP SIMD 在此基础上尝试利用指令级并行。三者都共享内存，差异主要来自编程抽象、调度成本和编译器向量化能力，而不是数学算法不同。

8 MPI 分布式并行

【平时作业】 【融合重构】

8.1 输入广播和计时方式

rank 0 读取输入，再广播 n 、目标模数和两个输入数组。正式计时前后使用 `MPI_Barrier`，由 rank 0 输出完整并行区间的时间。该口径包含必要通信，并避免只测某个先完成进程的局部时间。

8.2 CRT 模数级 MPI 并行

四个 rank 分别负责一个小模数，非零 rank 完成后把 residue 发送给 rank 0:

CRT 模数级 MPI 并行

```

1 for (int mod_id = 0; mod_id < CRT_CNT; ++mod_id) {
2     int owner = mod_id % size;
3     if (rank == owner) {
4         vector<u64> local;
5         multiply_mod_ntt(a, b, local, n, CRT_MODS[mod_id]);
6         if (rank == 0) residues[mod_id].swap(local);
7         else MPI_Send(local.data(), result_len, MPI_UINT64_T,
8             0, 100 + mod_id, MPI_COMM_WORLD);
9     }
10    if (rank == 0 && owner != 0) {
11        residues[mod_id].resize(result_len);
12        MPI_Recv(residues[mod_id].data(), result_len, MPI_UINT64_T,
13            owner, 100 + mod_id, MPI_COMM_WORLD,
14            MPI_STATUS_IGNORE);
15    }
16 }

```

每个进程在较长时间内独立执行完整 NTT，通信只发生在末尾，因此计算通信比高。缺点是并行度上限受 CRT 模数数量限制；当进程数超过 4 时，多余 rank 没有模数任务。

8.3 NTT stage 级 MPI 并行

stage 方案把当前层的 block 分给不同 rank，并在每层后同步完整数组：

stage 级 MPI 蝶形与层间同步

```

1 int block_l = blocks * rank / size;
2 int block_r = blocks * (rank + 1) / size;
3
4 #pragma omp parallel for schedule(static)
5 for (int block = block_l; block < block_r; ++block) {
6     int base = block * len;
7     #pragma omp simd
8     for (int k = 0; k < half; ++k) {
9         u64 x = f[base + k];
10        u64 y = mul_mod_ntt(f[base + k + half], roots[k], mod);
11        f[base + k] = add_mod(x, y, mod);
12        f[base + k + half] = sub_mod(x, y, mod);
13    }
14 }
15
16 MPI_Allgather(f.data() + displs[rank], counts[rank],
17     MPI_UINT64_T, gathered.data(), counts.data(),
18     displs.data(), MPI_UINT64_T, MPI_COMM_WORLD);
19 f.swap(gathered);

```

MPI_Allgather 使每个 rank 在下一层开始前都得到完整数组。该方案深入到单模数 NTT 内部，但一次卷积包含三次 NTT，每次有 $\log_2 N$ 层，因此产生约 $3\log_2 N$ 次完整数组集合通信，通信成本很容易超过计算分摊收益。

8.4 Barrett、DIT/DIF 与混合并行

v3 用 Barrett 降低本地模乘开销，v4 再用 DIF/DIT 删除显式位逆序；这些优化改善了 stage 系列的计算部分，却不能减少 Allgather 次数。v5 回到低通信的模数级 MPI，在每个 rank 内继续使用 OpenMP/SIMD 和 DIF/DIT，形成“进程间粗粒度、进程内细粒度”的混合路径。

9 GPU 并行优化：HIP 实验

【平时作业】 【融合重构】

9.1 一个线程负责一个蝶形

长度为 N 的每层有 $N/2$ 个蝶形，HIP 代码以全局线程号确定蝶形位置：

GPU 蝶形 kernel

```
1 __global__ void butterfly_kernel(u32* data,
2     const u32* twiddles, int n, int length, u32 mod) {
3     int tid = blockIdx.x * blockDim.x + threadIdx.x;
4     if (tid >= n / 2) return;
5
6     int half = length / 2;
7     int group = tid / half;
8     int j = tid % half;
9     int first = group * length + j;
10    int second = first + half;
11    int tw = j * (n / length);
12
13    u32 left = data[first];
14    u32 right = (u64)data[second] * twiddles[tw] % mod;
15    data[first] = left + right >= mod
16        ? left + right - mod : left + right;
17    data[second] = left >= right
18        ? left - right : left + mod - right;
19 }
```

group 和 j 把线性线程号映射到蝶形组和组内位置。每个线程只写两个独占元素，因此同一 stage 无写冲突。不同 workgroup 无法用块内屏障完成全局同步，所以普通实现每层启动一个 kernel。

9.2 旋转因子预计算

CPU 预先生成正、逆旋转因子表并复制到 GPU 全局内存，kernel 通过 $j*(n/\text{length})$ 查表。这避免每个 GPU 线程执行快速幂。预处理和首次传输应与 steady-state kernel 时间分开报告，否则无法判断查表的真实收益。

9.3 GPU Barrett 与 Montgomery

GPU Barrett 使用 __umul64hi 取得 64 位乘法高位；Montgomery 使用低 32 位乘法和右移：

GPU Montgomery 模乘

```
1 __host__ __device__ __forceinline__
2 u32 montgomery_reduce(u64 value) {
```

```

3   u32 m = static_cast<u32>(value) * MONT_NINV;
4   u64 reduced =
5       (value + static_cast<u64>(m) * MOD) >> 32;
6   if (reduced >= MOD) reduced -= MOD;
7   return static_cast<u32>(reduced);
8 }
9 __host__ __device__ __forceinline__
10 u32 montgomery_mul(u32 left, u32 right) {
11     return montgomery_reduce((u64)left * right);
12 }

```

输入、旋转因子和逆变换缩放因子统一进入 Montgomery 域，整个 NTT 中不反复转换。独立模乘实验可以突出规约差异，但完整 NTT 还包含位逆序、旋转因子读取、全局访存和多次 kernel 启动，因此整体加速会被其他开销稀释。

9.4 LDS 分块

平时 GPU 实验以 128 线程处理 256 元素，把前 8 个 stage 融合进一个 workgroup，并在 LDS 中使用 `__syncthreads()`。虽然减少了全局 kernel 启动，但增加了 LDS 搬运和块内屏障；实测 tiled 版本反而变慢。该结果表明共享内存不是“使用后必然加速”，必须同时考虑同步和 occupancy。

Part III

在共同算法与并行优化基础上的新增内容

10 统一 NTT 算法框架与实验基准

【本次新增】

10.1 计划生成与执行分离

四份平时作业分别生成位逆序表和旋转因子，导致各版本的计时范围不统一。为此，本次新增 `Plan`，提前统一生成这些数据。之后各并行后端只负责执行蝶形运算，使性能比较更加公平，也避免了重复预处理。

统一 NTT 计划结构

```

1 struct Plan {
2     size_t n;
3     Modulus modulus;
4     vector<size_t> bit_reverse;
5     vector<vector<uint32_t>> forward_twiddles;
6     vector<vector<uint32_t>> inverse_twiddles;
7
8     Plan(size_t size, Modulus mod);
9 };

```

`n` 和 `modulus` 定义数学问题；`bit_reverse` 固定排列；两个 `twiddle` 表消除正逆变换执行期间的快速幂和递推依赖。这样可以分别测量计划构建和 steady-state 变换，也可确保不同后端使用完全相同的旋转因子。

10.2 统一后端抽象

CPU 程序把“蝶形由谁执行”封装为后端枚举：

统一 CPU 后端分派

```

1 enum class Backend { Serial, Pthread, OpenMP, Simd, Hybrid };
2
3 void transform(vector<u32>& a, bool inverse,
4               const Plan& plan, Backend backend,
5               int threads) {
6     switch (backend) {
7         case Backend::Serial:
8             transform_serial(a, inverse, plan); break;
9         case Backend::Pthread:
10            transform_pthread(a, inverse, plan, threads); break;
11        case Backend::OpenMP:
12            transform_openmp(a, inverse, plan, threads); break;
13        case Backend::Simd:
14            transform_simd(a, inverse, plan); break;
15        case Backend::Hybrid:
16            transform_hybrid(a, inverse, plan, threads); break;
17    }
18 }

```

这里的意义不是强行把 MPI、CPU 与 HIP GPU 编译成同一个程序，而是统一数学输入、计划生成、计时范围和结果格式。各类后端可以保留独立的可执行文件，但必须遵守相同的变换流程和 CSV 字段。

10.3 统一卷积流程

所有后端共享的卷积流程

```

1 transform(A, false, plan, backend, threads);
2 transform(B, false, plan, backend, threads);
3
4 for (size_t i = 0; i < n; ++i)
5     A[i] = mul_mod(A[i], B[i], modulus);
6
7 transform(A, true, plan, backend, threads);

```

统一流程避免“某版本计入旋转因子生成、某版本不计”“某版本额外执行 CRT、另一版本只做单模数 NTT”等不公平比较。需要 CRT 时，四个小模数后端也必须以相同输入、相同输出长度和相同合并公式运行。

10.4 统一结果格式和基本正确性保证

程序统一输出后端、 N 、线程/rank/block 数、重复次数、均值、标准差和正确性标志。小规模样例与朴素卷积逐元素比较，正式并行版本与串行 NTT 比较，CRT 运行前检查模数乘积覆盖上界。

11 跨层次性能开销模型

【本次新增】

11.1 统一总时间分解

把任一后端的端到端时间写为

$$T = T_{arith} + T_{memory} + T_{sync} + T_{comm} + T_{launch} + T_{transfer} + T_{plan}.$$

其中 T_{arith} 是蝶形算术, T_{memory} 是缓存或全局内存访问, T_{sync} 是线程/barrier, T_{comm} 是 MPI 通信, T_{launch} 是 GPU kernel 启动, $T_{transfer}$ 是 H2D/D2H, T_{plan} 是旋转因子等预处理。不同架构并非简单改变同一项, 而是同时减少和增加不同项。

11.2 SIMD 与共享内存线程模型

SIMD 近似为

$$T_{SIMD} \approx \frac{T_{vectorizable}}{W} + T_{scalar} + T_{pack/reduce},$$

其中 W 为向量宽度, $T_{pack/reduce}$ 表示 64 位乘积拆分、lane 重排和 Montgomery 规约。因此理论 4 路 NEON 不等于 4 倍加速。

Pthread/OpenMP 可写为

$$T_{thread}(P) \approx \frac{T_{compute}}{P} + \log_2 N \cdot T_{barrier} + T_{runtime} + T_{bandwidth}.$$

当 N 小时固定开销占主导; 当 N 大时, 多线程可能转为内存带宽受限, 继续增加线程数也不会线性加速。

11.3 MPI 两种粒度模型

CRT 模数级方案近似为

$$T_{mod-MPI} \approx T_{single-modulus} + T_{broadcast} + T_{gather} + T_{CRT}.$$

四个模数独立, 主要通信只发生在输入广播和 residue 汇总。

stage 级方案近似为

$$T_{stage-MPI} \approx \frac{T_{compute}}{P} + 3 \log_2 N (\alpha + \beta N),$$

其中 α 为集合通信启动延迟, β 为单位数据传输时间。三次 NTT 的逐层 Allgather 使通信项随 $N \log N$ 增长, 解释了 stage 方案实测明显慢于模数级方案。

11.4 GPU 端到端模型

GPU 应区分

$$T_{kernel} = T_{bit-reverse} + T_{butterfly} + T_{pointwise} + T_{scale}$$

与

$$T_{GPU,end} = T_{plan} + T_{H2D} + T_{kernel} + T_{D2H}.$$

独立模乘只反映 T_{arith} ; 完整 NTT 还受位逆序、全局访存和 $O(\log N)$ 次 kernel 启动影响。若数据已驻留 GPU, 则 T_{H2D} 和 T_{D2H} 可以由上下游计算摊薄; 若仅从 CPU 发起一次小任务, 传输可能使 GPU 失去优势。

12 跨架构评价指标设计

【本次新增】

12.1 为什么不能直接排列原始时间

SIMD、Pthread/OpenMP、MPI 和 HIP 平时实验来自不同 CPU/GPU 与计时边界。把 42.22 ms、67.734 ms 和 1.1648 ms 直接排列并不能说明哪种编程模型更好，因为硬件、模数数量和计时内容不同。跨架构分析应以各平台内部串行基准和端到端开销为中心。

12.2 归一化指标

本文采用：

- 加速比 $S = T_{serial}/T_{parallel}$ ；
- 并行效率 $E = S/P$ ；
- 核心时间与端到端时间之比；
- 单次蝶形平均时间（同模数、同流程时）；
- 计算、同步、通信、启动和传输时间占比；
- 不同规模下后端性能曲线的交点。

12.3 消融实验逻辑

每个平台按单一变量递进：SIMD 从普通 NTT 到 Montgomery、NEON、DIT/DIF；共享内存从串行 CRT 到 Pthread、OpenMP 模数并行、OpenMP 内部并行和 OpenMP SIMD；MPI 从串行基准到模数级、stage、Barrett、DIT/DIF 和混合同行；GPU 从 runtime modulo 到 Barrett、Montgomery 和 LDS tiled。新增 Lazy Reduction 复测固定模乘方法和并行配置，只改变蝶形值域与规约时机。这样的版本链能把性能变化归因于具体设计，而不是只比较“最慢版本”和“最快版本”。

Part IV

统一性能测试与综合分析

13 实验环境与统一测试方法

13.1 各平台环境

表 2: 四类平时实验的运行环境

实验	平台	主要配置	工具链
SIMD	ARM/aarch64 服务器	NEON, NTT 长度主要为 131072	C++, -O2/-O3
Pthread/ OpenMP	OpenEuler ARM 服务器	每任务 1 个 8 核 CPU	Pthread、OpenMP
MPI	ARM 集群	4 rank, rank 内可用 OpenMP	mpic++ -O3 -fopenmp
GPU	AMD Ryzen AI MAX+ PRO 395 / Radeon 8060S	RDNA 3.5, 40 CU, wave32, 64 KB LDS/CU	HIP 7.13, AMD Clang, rocprofv3

历史实验的硬件不同, 因此表 2 既是环境说明, 也是跨架构比较的限制条件。本文不把不同机器的绝对毫秒数当作直接排名。

13.2 统一实验规模与输入

统一复测建议选择

$$N = 2^{10}, 2^{14}, 2^{17}, 2^{18}, 2^{20}, 2^{22},$$

并使用固定随机种子生成相同输入。单模数实验使用 998244353; 大模数 CRT 实验使用相同的四个 NTT 友好质数, 并保证模数乘积覆盖系数上界。小规模样例可与朴素卷积对照, 错误结果不进入性能统计, 但不再设置独立正确性章节。

13.3 计时边界

CPU core 时间覆盖两次正变换、点值乘和一次逆变换; MPI 端到端时间还覆盖输入广播、residue 汇总和 CRT 合并, 并取所有 rank 的最大完成时间; GPU 同时报告纯 kernel 时间和 H2D/kernel/D2H 端到端时间。旋转因子生成、内存分配和首次上下文初始化单独列出, 避免混用 steady-state 与首次调用口径。

13.4 重复实验与命令

每个配置先预热, 再正式运行至少 5-10 次, 报告均值和标准差。平时 MPI 实验使用:

MPI 实验编译和运行命令

```
1 mpic++ -std=c++17 -O3 -fopenmp main.cc -o main
2 mpiexec -np 4 -machinefile $PBS_NODEFILE ./main 4 4 5
```

统一项目使用:

统一复测命令

```

1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
2 cmake --build build -j
3 ctest --test-dir build --output-on-failure
4 ./scripts/run_cpu.sh
5 OMP_NUM_THREADS=2 ./scripts/run_mpi.sh

```

14 各体系结构内部性能分析

【平时作业】 【融合重构】

14.1 SIMD 版本消融实验

表 3: $N = 131072$ 时 SIMD 平时作业结果 (单位: ms)

模数	普通 NTT	加减 NEON	模乘 NEON	DIT/DIF
7340033	55.5610	47.9239	50.0162	42.2464
104857601	59.6159	48.4696	50.1707	41.9330
469762049	62.7503	48.5684	50.3972	42.4692
平均	59.31	48.32	50.19	42.22

Montgomery 与加减 NEON 将平均时间从 59.31 ms 降至 48.32 ms, 约加速 1.23 倍。继续向量化模乘后平均为 50.19 ms, 反而略慢, 说明 64 位乘积拆分和旋转因子装载存在成本。DIT/DIF 将平均时间进一步降至 42.22 ms, 相对普通 NTT 约 1.41 倍, 证明减少位逆序访存比单独追求更宽模乘更有效。

14.2 Pthread/OpenMP 版本消融实验

表 4: 共享内存四模数 CRT 版本结果

版本	核心时间/ms	相对串行加速比
四模数 CRT 串行	927.15	1.00
四模数 CRT Pthread	318.32	2.91
OpenMP 模数并行	251.48	3.69
OpenMP NTT 内部并行	353.28	2.63
OpenMP + SIMD	144.90	6.40

Pthread 手工线程版本明显优于串行 CRT, 但 OpenMP 模数并行更快, 因为四个完整 NTT 任务粒度粗、同步少。OpenMP 内部并行每层进入并行区并隐式同步, 性能不如模数并行。OpenMP+SIMD 同时减少模数间串行和局部指令成本, 在该组旧实验中最好。这里的 OpenMP+SIMD 属于平时作业, 不计为新增内容。

14.3 MPI 版本消融实验

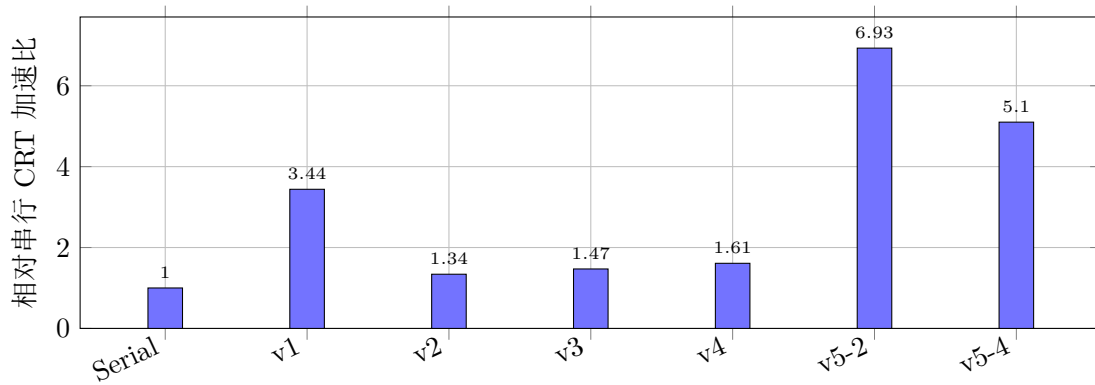


图 1: MPI 第 4 号大规模样例的版本消融结果

串行 CRT 平均用时为 469.360 ms, v1 的模数级并行达到 3.44 倍, 而逐层同步的 v2 只有 1.34 倍, 说明 `Allgatherv` 是主要瓶颈。Barrett 和 DIT/DIF 使 v2 依次改善约 8.7% 和 8.9%, 但没有改变通信结构。v5 在 $4 \text{ rank} \times 2$ 线程时用时 67.734 ms、加速 6.93 倍; 增加到每 rank 4 线程反而降至 5.10 倍, 表明线程数继续增加后, 同步和内存竞争抵消了计算收益。

14.4 HIP 版本性能分析

固定 $N = 2^{20}$ 时, 平时 HIP 实验得到:

表 5: 完整 HIP NTT 的模乘和 LDS 优化

方法	正变换/ms	逆变换/ms	正变换相对 baseline
Runtime modulo	1.1523	0.7766	1.000
Barrett	1.1348	0.7522	1.015
Montgomery	1.1506	0.7504	1.002
Montgomery tiled	1.2047	0.7791	0.957

独立模乘中 Barrett 和 Montgomery 相对 runtime modulo 分别加速 2.156 和 5.350 倍, 但完整 NTT 只改善约 1.5% 和 0.2%, 说明模乘不是唯一瓶颈。LDS tiled 慢约 4.3%, 其搬运和屏障成本超过减少 kernel 启动的收益。

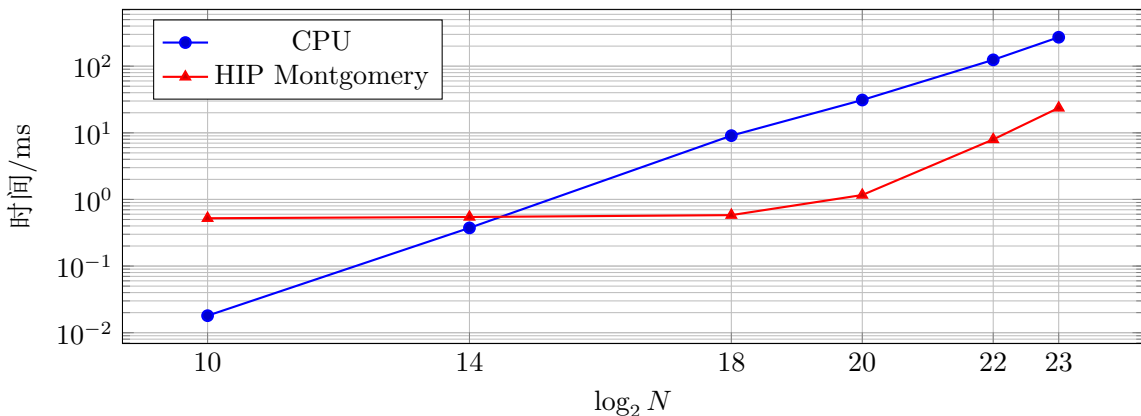


图 2: CPU 与 HIP NTT 的规模扩展曲线 (纵轴为对数刻度)

$N \leq 2^{14}$ 时 GPU kernel 启动占主导, CPU 更快; 从 2^{18} 开始 GPU 获得明显优势; 2^{20} 达到该组测试最大加速比 26.66; 再增大后, 工作集和跨步访存使加速比回落。

15 跨架构综合分析

【融合重构】 【本次新增】

15.1 跨机器原始时间的可比性

ARM SIMD、OpenEuler 线程实验、MPI 集群和 AMD GPU 的硬件、模数数量与计时边界不同, 因此不能根据原始毫秒数直接断言 GPU、MPI 或 SIMD 谁“绝对最好”。例如 SIMD 表是单模数 NTT, 线程与 MPI 表是四模数 CRT, 而 GPU 表主要为单次变换纯 kernel; 这些工作量并不相同。

15.2 归一化性能总结

表 6: 各平台内部代表性加速趋势

平台	代表版本	平台内加速比	主要收益来源
ARM SIMD	NEON+DIT/DIF	约 1.41	模加减向量化和删除位逆序
Pthread/OpenMP	OpenMP+SIMD CRT	6.40	模数级并行和局部向量化
MPI	v5, 4 rank $\times$ 2 线程	6.93	低通信模数并行 +rank 内优化
AMD HIP	$N = 2^{20}$ Montgomery	26.66	GPU 大规模蝶形吞吐

表 6 只能说明“每个平台相对自身基准取得了多少改善”。HIP 的 26.66 还使用单线程 CPU 作为分母; 若与多线程 CPU 端到端卷积比较, 加速比会改变。

15.3 规模扩展性

小规模时 SIMD/线程/GPU 的固定开销占比较高; 规模增大后 $O(N \log N)$ 计算逐渐覆盖固定开销; 继续增大又可能受到 cache、DRAM、显存或通信带宽限制。因此性能曲线通常先改善后趋于平坦, GPU 加速比甚至会在超出缓存后回落。

15.4 优化不能线性叠加的原因

SIMD 可能被 64 位模乘和内存带宽限制; OpenMP 可能被层间 barrier 和运行时调度限制; MPI 可能被集合通信限制; GPU 可能被位逆序、全局访存和 kernel 启动限制。Lazy Reduction 只减少算术规约, 若瓶颈位于通信或访存, 整体收益仍会受限。这也解释了 NEON 模乘、MPI Barrett 和 GPU Montgomery 在不同场景中的收益差异。

15.5 基于问题特征的 NTT 后端选择策略

【本次新增】

选择因素包括 N 、批次数量 B 、是否需要 CRT、数据是否已经在 GPU、可用 CPU 核心/MPI 节点以及任务更关注延迟还是吞吐。

表 7: NTT 后端选择建议

场景	推荐后端	原因
小规模、单次调用	serial/SIMD	避免线程和 GPU 固定开销
中等规模、单节点	OpenMP 或 OpenMP SIMD	蝶形足以覆盖 barrier, 且共享内存通信低
CRT 大模数、单节点	OpenMP 模数并行	四个完整 NTT 独立, 调度简单
CRT 大模数、多 rank	MPI 模数级并行	通信集中在末尾, 计算通信比高
单模数大规模	GPU	大量规则蝶形提供高吞吐
大批量卷积	GPU/任务级并行	摊薄初始化、启动和传输
数据已驻留 GPU	GPU	H2D/D2H 不再是每次固定成本
CPU 数据、单次中小任务	SIMD/OpenMP	GPU 端到端时间可能更高

若需要 CRT, 首先进入模数级 OpenMP/MPI 路径; 否则根据规模和数据驻留判断 CPU 或 GPU。决策流程可表达为:

后端选择的分析性决策流程

```

1  if (requires_CRT)
2      backend = multi_node ? MPI_modulus : OpenMP_modulus;
3  else if (data_on_GPU && (large_N || large_batch))
4      backend = GPU;
5  else if (small_N)
6      backend = serial_or_SIMD;
7  else if (medium_N)
8      backend = OpenMP_or_OpenMP_SIMD;
9  else
10     backend = compare_GPU_end_to_end_with_OpenMP;

```

批量 B 增大时, GPU 每任务平均时间近似为

$$\bar{T}_{GPU} = \frac{T_{init} + B T_{compute}}{B},$$

固定初始化被摊薄, 因此 GPU 的适用下限下降。数据已经在显存时同理。最终阈值应由目标硬件实测曲线确定, 而不是写死为某个 2^k 。

16 实验局限与改进方向

- 历史数据来自不同机器与计时边界, 只能解释各平台内部趋势;
- GPU 平时数据来自 AMD HIP;
- OpenMP simd 是否生成高效指令依赖编译器, 本报告不新增显式 SIMD 对比实验;
- CRT 模数级 MPI 的并行度受模数数量限制, 更多 rank 应用于批次而非空闲;
- stage MPI 复制完整数组, 后续可研究分布式 Stockham 或六步 NTT;
- GPU tiled 参数依赖具体硬件, 可继续研究 warp shuffle、Stockham 和 GPU-aware MPI;

17 新增内容说明

表 8: 本次新增内容及其与平时作业的区别

新增内容	所在章节	与平时作业的区别
统一 NTT 计算图与跨架构映射	第 4 章	将四类实现放入同一层次结构, 比较并行对象和同步位置
Lazy Reduction 与值域控制	第 3 章	延迟蝶形标准化, 并针对 32/64 位数据类型证明安全范围
统一 NTT 算法框架与实验基准	第 10 章	统一 Plan、输入、计时、输出和基本正确性保证
跨层次性能开销模型	第 11 章	同时建模算术、访存、同步、通信、启动和传输
跨架构评价指标	第 12 章	不再直接排列跨机器原始时间, 改用归一化指标
后端选择策略	第 15 章	根据规模、批次、CRT 和数据驻留综合选择后端

新增工作建立在“共同算法基础”和“各体系结构优化”两个板块之上, 既增加 Lazy Reduction 算法及值域分析, 也把旧实验转换为统一、可解释的跨架构研究。

18 总结

本文围绕同一 NTT 卷积计算图, 融合讨论了 SIMD、Pthread/OpenMP、MPI 与 GPU。算法上的共同核心是 stage 内独立蝶形、stage 间依赖和高频模乘; 体系结构差异主要体现在并行粒度、同步机制、数据位置和固定开销。

平时实验表明, SIMD 的收益受模乘拆分限制, DIT/DIF 通过减少位逆序访存取得进一步改善; Pthread/OpenMP 中粗粒度 CRT 模数并行优于频繁同步的内部并行; MPI 中低通信模数级方案明显优于逐 stage 全数组同步, rank 内线程数也不是越多越好; GPU 适合中大规模规则蝶形, 但独立模乘的巨大加速不会直接等于完整 NTT 加速, LDS 分块也可能因同步与搬运变慢。

本次新增 Lazy Reduction、统一 Plan、跨层次开销模型、归一化评价和后端选择策略。Lazy Reduction 通过扩大合法中间值域减少规约次数, 但实际收益仍需服务器与 HIP 平台按统一口径复测。综合来看, NTT 优化的关键不是最大化并行单元数量, 而是同时控制算术、同步、通信与访存开销: 小规模优先 serial/SIMD, 中等规模使用共享内存线程, 大模数 CRT 优先模数级 OpenMP/MPI, 大规模或批量且数据适合驻留显存时使用 HIP GPU。

感谢老师和助教这一学期的付出, 祝福老师和助教工作顺利, 假期愉快!